

INTERVIEW-READY REFERENCE GUIDE

SQL — Basics to Advanced

A complete, example-driven path through SQL — from SELECT to window functions, CTEs, indexing, and the questions interviewers actually ask.

```
-- the journey, in one line
SELECT * FROM beginner
JOIN intermediate ON ready = true
JOIN advanced ON confident = true
JOIN interview_ready ON practiced = true;
```

Prepared for: Tharun | Data Analyst / Data Scientist Interview Prep

1. Selecting & Filtering Data

SELECT — Choose Columns

Pulls specific columns from a table. Use * to get every column.

```
SELECT first_name, last_name
FROM employees;

-- get everything
SELECT * FROM employees;

-- DISTINCT
SELECT DISTINCT department FROM employees;
```

WHERE — Filter Rows

Keeps only the rows that match a condition. Runs before grouping or sorting.

```
SELECT * FROM employees
WHERE department = 'Sales' AND salary > 50000;
```

Comparison Operators

Operator	Meaning
=	equal to
!= or <>	not equal to
> / <	greater than / less than
BETWEEN	within a range
IN (...)	matches a list
LIKE	pattern match
IS NULL	empty value check

ORDER BY — Sort Results

Sorts the output. The default is ascending (smallest to largest).

```
SELECT name, age
FROM users
ORDER BY age DESC;

-- MySQL / PostgreSQL
LIMIT 10;

-- SQL Server
TOP 10;

-- Oracle
FETCH FIRST 10 ROWS ONLY;
```

TIP Use *LIMIT 10* to cap how many rows come back.

Q1. Why doesn't LIMIT work the same way across all databases?

Each vendor standardized row-capping differently before the SQL standard caught up: MySQL/PostgreSQL use LIMIT, SQL Server uses TOP, and Oracle (12c+) uses FETCH FIRST n ROWS ONLY.

Q2. What's the difference between WHERE and a comparison inside SELECT?

WHERE decides which rows survive before the result set is built; conditions inside SELECT (like CASE) shape how a surviving row is displayed — they don't remove rows.

2. Combining Tables with JOIN

JOINS let you pull related data from two (or more) tables at once, by matching a shared column — usually an ID.

INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN

```
SELECT orders.id, customers.name
FROM orders
JOIN customers ON orders.customer_id = customers.id;
```

Join Type	Behavior
INNER JOIN	Only rows that match in both tables.
LEFT JOIN	All rows from the left table, matched where possible.
RIGHT JOIN	All rows from the right table, matched where possible.
FULL OUTER JOIN	Every row from both tables, matched or not.

Quick Example

```
-- LEFT JOIN: keep all customers, even with no orders
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id;
```

TIP Always alias your tables (c, o) once you join more than one — it keeps queries readable.

Q3. How is a LEFT JOIN different from an INNER JOIN when there's no match?

INNER JOIN drops the row entirely if there's no match on the right table. LEFT JOIN keeps the left row anyway and fills the right side's columns with NULL.

Q4. Can you get the same result as a FULL OUTER JOIN in MySQL, which doesn't support it?

Yes — UNION a LEFT JOIN with a RIGHT JOIN (or a LEFT JOIN with the tables swapped), then de-duplicate with UNION instead of UNION ALL.

3. Grouping & Aggregating

Aggregate Functions

Summarize many rows into a single value.

Function	Returns
COUNT()	number of rows
SUM()	total of a numeric column
AVG()	average value
MIN() / MAX()	smallest / largest value

GROUP BY — Bucket Rows Together

Groups rows that share a value, so aggregates calculate per-group instead of across the whole table.

```
SELECT department, COUNT(*) AS total_emp
FROM employees
GROUP BY department;
```

HAVING — Filter Groups

WHERE filters rows before grouping. HAVING filters groups after aggregation.

```
SELECT department, COUNT(*) AS total_emp
FROM employees
GROUP BY department
HAVING COUNT(*) > 10;
```

TIP Rule of thumb — filter rows with *WHERE*. Filter groups or aggregates with *HAVING*.

Q5. Why can't you put an aggregate condition in WHERE?

WHERE runs before the aggregation step, so aggregate values like COUNT(*) don't exist yet at that point — HAVING runs after GROUP BY, once the aggregates are computed.

Q6. Does COUNT(*) treat NULLs differently from COUNT(column_name)?

COUNT(*) counts every row regardless of NULLs. COUNT(column_name) only counts rows where that specific column is not NULL.

4. Adding, Changing, Removing Data

INSERT — Add New Rows

```
INSERT INTO employees (name, department, salary)
VALUES ('Asha', 'Marketing', 45000);
```

UPDATE — Change Existing Rows

Always pair with WHERE — without it, every row gets updated.

```
UPDATE employees
SET salary = 55000
WHERE id = 12;
```

DELETE — Remove Rows

Same rule applies — DELETE without WHERE wipes the whole table.

```
DELETE FROM employees
WHERE id = 12;
```

CREATE TABLE — Define Structure

```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  department VARCHAR(50),
  salary DECIMAL(10,2)
);
```

CAUTION Before running UPDATE or DELETE in production, run the same WHERE clause inside a SELECT first to confirm which rows are affected.

Q7. What's the difference between DELETE, TRUNCATE, and DROP?

DELETE removes rows one at a time (can use WHERE, logged, can be rolled back), TRUNCATE removes all rows at once and resets identity counters (minimally logged, faster), and DROP removes the entire table structure along with its data.

Q8. What's the difference between a PRIMARY KEY and a UNIQUE key?

A table can have only one PRIMARY KEY and it disallows NULLs; a table can have multiple UNIQUE constraints, and most databases allow one NULL per UNIQUE column (some allow more).

5. How a Query Actually Runs

You write a query top to bottom, but the database executes it in a different order. Knowing this order explains a lot of "why doesn't this work" moments — and it's the foundation for everything more advanced later in this guide.

```
1. FROM -> 2. JOIN -> 3. WHERE -> 4. GROUP BY ->
5. HAVING -> 6. SELECT -> 7. ORDER BY -> 8. LIMIT
```

- FROM / JOIN — the database first figures out which table(s) to pull data from and combines them.
- WHERE — rows that don't match the condition get thrown out, before any grouping happens.
- GROUP BY — remaining rows are bucketed together by the column(s) you specify.
- HAVING — entire groups are filtered out based on aggregate conditions.
- SELECT — only now does it figure out which columns or expressions to actually return.
- ORDER BY — the remaining result set gets sorted.
- LIMIT — finally, the row count is capped.

TIP *This is why you can't use a column alias from SELECT inside a WHERE clause — WHERE runs before SELECT even exists yet.*

Q9. Interviewers love this one: "In what order does a SQL query actually execute?" How do you answer confidently?

State the logical processing order — FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT — and give the one-line reason it matters: aliases created in SELECT aren't visible to WHERE or GROUP BY (in most databases) because those clauses run first.

6. Subqueries — Bridging to Advanced

A subquery is a query nested inside another query. Once SELECT, WHERE, JOIN, and GROUP BY feel comfortable, this is the natural next step.

Scalar Subquery — Returns One Value

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Subquery with IN / EXISTS

```
-- IN: customer ids that appear in orders
SELECT * FROM customers
WHERE id IN (SELECT customer_id FROM orders);
```

```
-- EXISTS: often faster on large tables
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.id
);
```

Correlated Subquery — Depends on the Outer Row

Runs once per outer row, referencing a column from that row.

```
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(salary) FROM employees
    WHERE department = e.department
);
```

TIP A subquery answers a question once. If you find yourself repeating the same subquery, a CTE (next topic) is usually cleaner.

Q10. Why is EXISTS often faster than IN on large tables?

EXISTS can stop as soon as it finds one matching row, while IN typically needs the full subquery result materialized first — the gap narrows on modern optimizers, but EXISTS is still the safer default with NULLs and large sets.

Q11. What happens if the subquery in an IN clause returns a NULL?

If any value in the IN list is NULL, rows that don't match any non-NULL value can silently be excluded from the result — a classic "NOT IN returns nothing" bug. EXISTS avoids this trap.

7. CTEs — Common Table Expressions

A CTE is a named, temporary result set defined with WITH, that you can reference later in the query like a table. It exists only for the duration of that one query.

Basic Syntax

```
WITH high_earners AS (
    SELECT * FROM employees WHERE salary > 80000
)
SELECT department, COUNT(*)
FROM high_earners
GROUP BY department;
```

Why a CTE Over a Subquery?

- Readable top-to-bottom, instead of nested inside-out.
- Can be referenced more than once in the same query.
- Easy to build step-by-step, testing each piece.

Multiple CTEs

```
WITH a AS (  
    SELECT ...  
),  
b AS (  
    SELECT ... FROM a  
)  
SELECT * FROM b;
```

Recursive CTE — For Hierarchies

A CTE that references itself — used for org charts, category trees, or any parent-child structure.

```
WITH RECURSIVE org_chain AS (  
    SELECT id, name, manager_id FROM employees  
    WHERE manager_id IS NULL -- anchor: top of chain  
    UNION ALL  
    SELECT e.id, e.name, e.manager_id  
    FROM employees e  
    JOIN org_chain o ON e.manager_id = o.id -- recursive step  
)  
SELECT * FROM org_chain;
```

TIP A recursive CTE always needs an anchor query (the base case) and a recursive query joined back to itself.

Q12. When would you reach for a CTE instead of a subquery, in an interview answer?

When the same derived result is needed more than once in a query, when nesting would get too deep to read, or when the logic is naturally sequential — build result A, then B from A, then C from B.

Q13. What's the difference between a CTE and a temporary table (or view)?

A CTE exists only for the single statement it's defined in and isn't stored anywhere; a temp table physically stores data for the session and can be indexed; a view is a stored, reusable query definition that persists across sessions.

8. Window Functions — The Basics

A window function calculates across a set of rows related to the current row — without collapsing them into a single result, the way GROUP BY does. Every input row still gets a row back.

The Anatomy of OVER()

```
FUNCTION() OVER (
  PARTITION BY column  -- splits rows into groups
  ORDER BY column      -- orders rows within each group
)
```

TIP *PARTITION BY* is the window-function version of *GROUP BY* — except rows aren't collapsed.

GROUP BY vs. Window Function

GROUP BY	Window Function
Collapses rows into one per group	Keeps every original row
Can't see individual row detail	Shows detail and aggregate together
SELECT limited to grouped columns	SELECT any column, plus the window calc

Example: Department Average, Alongside Each Employee

```
SELECT name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees;
```

Q14. How would you explain a window function to someone who only knows GROUP BY?

GROUP BY answers "what's the total per group" but hides the individual rows; a window function answers the same question while still showing you every row — each employee's own salary next to their department's average, in the same row.

9. Ranking & Positional Functions

The Three Ranking Functions

Function	Behavior on Ties
ROW_NUMBER()	always unique, no ties — 1, 2, 3, 4
RANK()	ties share a rank, then skips — 1, 2, 2, 4
DENSE_RANK()	ties share a rank, no skip — 1, 2, 2, 3

Example: Rank Employees by Salary, per Department

```
SELECT name, department, salary,
       RANK() OVER (
```

```
        PARTITION BY department
        ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
```

LAG() & LEAD() — Look at Neighboring Rows

Pull a value from a previous (LAG) or following (LEAD) row, without a self-join.

```
SELECT order_date, amount,
       LAG(amount, 1) OVER (ORDER BY order_date) AS prev_amount,
       amount - LAG(amount, 1) OVER (ORDER BY order_date) AS change
FROM orders;
```

Common Use: Top N per Group

```
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY department ORDER BY salary DESC
    ) AS rn
    FROM employees
)
SELECT * FROM ranked WHERE rn <= 3;
```

Q15. Classic interview task: "Find the Nth highest salary." What's the cleanest modern approach?

Wrap the table in a CTE with `DENSE_RANK()` OVER (ORDER BY salary DESC), then filter WHERE rank = N in the outer query — `DENSE_RANK` is preferred over `ROW_NUMBER` here because it correctly handles tied salaries as the same rank.

Q16. How do `RANK()` and `DENSE_RANK()` differ when three people tie for 2nd place?

`RANK()` gives them all 2, then jumps to 5 for the next row (2, 2, 2, 5); `DENSE_RANK()` gives them all 2 and continues at 3 with no gap (2, 2, 2, 3).

10. Indexing & Query Performance

An index is a separate, sorted data structure that lets the database find rows without scanning the whole table — similar to a book's index pointing straight to a page.

Creating an Index

```
CREATE INDEX idx_customer_email
ON customers (email);
```

```
-- composite index, for queries filtering on both columns
CREATE INDEX idx_orders_cust_date
ON orders (customer_id, order_date);
```

Good Candidates for an Index

- Columns used in WHERE clauses often
- Columns used to JOIN tables
- Columns in ORDER BY on large tables
- Foreign key columns

The Tradeoff

- Faster SELECT queries
- Slower INSERT / UPDATE / DELETE — every index has to update too
- Extra storage space used

EXPLAIN — See How the Query Actually Runs

Shows whether the database used an index ("index scan") or read the whole table ("sequential / full table scan").

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
```

CAUTION *An index only helps if the database actually uses it. Functions wrapped around an indexed column (e.g. LOWER(email)) usually stop the index from being used.*

Q17. What's the difference between a clustered and a non-clustered index?

A clustered index determines the physical storage order of the table's rows — a table can have only one. A non-clustered index is a separate structure that points back to the row's location, and a table can have several.

Q18. Why wouldn't you index every column just to be safe?

Every index adds storage overhead and slows every INSERT, UPDATE, and DELETE (since each index needs updating too) — indexes should target columns that are actually filtered, joined, or sorted on frequently.

11. CASE, Sets & Full Syntax Reference

CASE — Conditional Logic Inside a Query

```
SELECT name, salary,
       CASE
         WHEN salary >= 80000 THEN 'Senior'
         WHEN salary >= 50000 THEN 'Mid'
         ELSE 'Junior'
       END AS level
FROM employees;
```

Set Operations

Operator	Behavior
UNION	combines, removes duplicates
UNION ALL	combines, keeps duplicates
INTERSECT	rows in both queries
EXCEPT	in first, not in second

NULL Handling

```
SELECT name,
       COALESCE(phone, 'N/A') AS phone
FROM customers;
```

The Full SELECT Statement, in Writing Order

```
SELECT column1, AGG_FUNC(column2)
FROM table_name
JOIN other_table ON table_name.id = other_table.id
WHERE condition
GROUP BY column1
HAVING aggregate_condition
ORDER BY column1 ASC|DESC
LIMIT n;
```

Quick Keyword Reference

DISTINCT · AS · AND · OR · NOT · IN · EXISTS · CASE · WHEN · COALESCE · WITH · OVER() · PARTITION BY · CREATE INDEX

TIP You've gone from *SELECT* to window functions and indexing — that's the full range most data roles use day to day.

Q19. Why use UNION ALL over UNION when you know there won't be duplicates?

UNION ALL skips the extra step of scanning for and removing duplicates, so it's cheaper — UNION should only be used when de-duplication is actually needed.

Q20. What does COALESCE do that a simple OR replacement can't?

COALESCE returns the first non-NULL value across any number of arguments in one expression, which handles NULL fallback logic that comparison operators (which themselves return NULL when compared against NULL) can't do cleanly.

2. Top SQL Interview Questions — Concepts

A quick-fire round for the questions that come up in almost every SQL interview, beyond what's already answered topic-by-topic above.

Q21. What are the ACID properties of a transaction?

Atomicity (all-or-nothing), Consistency (valid state to valid state), Isolation (concurrent transactions don't interfere), and Durability (once committed, changes survive a crash).

Q22. What's the difference between a primary key and a foreign key?

A primary key uniquely identifies each row in its own table and can't be NULL. A foreign key is a column in one table that references a primary key in another, enforcing referential integrity between tables.

Q23. What is normalization, and can you name the first three normal forms?

Normalization organizes data to reduce redundancy and avoid update anomalies. 1NF: atomic values, no repeating groups. 2NF: 1NF plus no partial dependency on part of a composite key. 3NF: 2NF plus no transitive dependency on non-key columns.

Q24. When would you denormalize a database on purpose?

When read performance matters more than write efficiency or storage — for example, reporting/analytics tables where joining many normalized tables on every query is too slow, so some redundancy is accepted for speed.

Q25. What's the difference between a stored procedure and a function?

A function must return a value and can be used inside a SELECT statement, but usually can't modify data. A stored procedure may or may not return a value, can perform DML/DDI, and is called on its own rather than embedded in a query.

Q26. What is a self join and when would you use one?

A self join joins a table to itself, typically to compare rows within the same table — like matching each employee to their manager, where both are stored in the same employees table.

Q27. What's the difference between WHERE 1=1 and no WHERE clause at all in dynamically built queries?

WHERE 1=1 is a placeholder that's always true, used so additional AND conditions can be appended programmatically without special-casing the first condition — functionally it behaves the same as having no filter.

Q28. How do you find duplicate rows in a table?

GROUP BY the columns that define a duplicate, then filter with HAVING COUNT(*) > 1 — this returns the values that appear more than once, which you can join back to the original table to see the full duplicate rows.

Q29. What's the difference between a view and a materialized view?

A regular view is just a stored query — it re-runs against live data every time it's queried. A materialized view stores the actual result set on disk, which is faster to read but needs to be refreshed to reflect new data.

Q30. Why might two seemingly identical queries have very different performance?

Differences in indexing, statistics/estimated row counts, how the optimizer chooses a query plan, data skew, or implicit type conversions on a filtered column can all cause the same logical query to run very differently in practice.

3. Write-It-Yourself Practice Problems

These are the hands-on tasks interviewers ask you to actually write on a whiteboard or shared screen. Try each one before checking the solution.

Assume an employees(id, name, department, salary, manager_id) table unless stated otherwise.

1. Second highest salary

Write a query to find the second-highest salary in the employees table, without hardcoding a value.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

2. Nth highest salary (general case)

Write a query that returns the Nth highest salary, where N is a parameter.

```
WITH ranked AS (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
)
SELECT DISTINCT salary FROM ranked WHERE rnk = N;
```

3. Employees earning more than their manager

Find every employee whose salary is higher than their own manager's salary.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

4. Departments with no employees

Given departments(id, name) and employees(id, department_id), list departments that currently have zero employees.

```
SELECT d.name
FROM departments d
LEFT JOIN employees e ON d.id = e.department_id
WHERE e.id IS NULL;
```

5. Running total of orders

Given orders(order_id, order_date, amount), show a running total of amount ordered by date.

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

6. Delete duplicate rows, keep one copy

Given employees with duplicate rows on every column, delete all but one copy of each duplicate.

```
WITH ranked AS (
  SELECT id, ROW_NUMBER() OVER (
    PARTITION BY name, department, salary ORDER BY id
  ) AS rn
  FROM employees
)
DELETE FROM employees
WHERE id IN (SELECT id FROM ranked WHERE rn > 1);
```

7. Month-over-month percentage change

Given monthly_sales(month, total), compute the percentage change in total from the previous month.

```
SELECT month, total,
       ROUND(
         (total - LAG(total) OVER (ORDER BY month))
         / LAG(total) OVER (ORDER BY month) * 100, 2
       ) AS pct_change
FROM monthly_sales;
```

TIP Say your plan out loud before typing: name the tables you need, the join condition, and whether you need GROUP BY, a window function, or a CTE — interviewers weight your reasoning as much as the final query.